

Generalized Linear Models

Denis Nedić

dn98183@student.uni-lj.si

Introduction

In Part 1 of this assignment, I implemented multinomial and ordinal logistic regression models using both a custom Python autograd engine and the `scipy.optimize` library, comparing their execution speed and numerical stability.

In Part 2, I applied the multinomial model to analyze feature significance in basketball shot selection and constructed a custom Data-Generating Process (DGP) to demonstrate the statistical advantage of ordinal regression over multinomial regression when predicting rare, ordered classes.

Part 1: Model Implementation

1. Numerical Stability & Implementation Challenges

During the development of the custom autograd engine, the model exhibited high sensitivity to floating-point limits, frequently resulting in an `OverflowError` due to exploding gradients. This occurred during the backward pass when calculating the derivative of the negative log-likelihood if the predicted probability p approached zero.

To resolve this, two mathematical safety nets were implemented: first, correcting the base of the custom sigmoid function to ensure values could not trigger a division by zero, and second, introducing an epsilon bound ($p = p + 10^{-15}$) prior to the log calculation. In contrast, the `scipy` implementation was highly stable. By defining explicit parameter bounds, it naturally avoided these floating point overflow errors without requiring manual mathematical clipping.

2. Speed and Optimization Mechanics

The optimization mechanics between the two implementations yielded a massive difference in execution time. The custom autograd engine required manual hyperparameter tuning (specifically the learning rate and the number of epochs) to prevent the weights from diverging.

Conversely, optimization using Scipy utilized the L-BFGS-B algorithm, a dynamically scaling quasi-Newton method. Because L-BFGS-B approximates the inverse Hessian matrix to intelligently adjust step sizes, it converged in a fraction of the iterations without the need for learning rate tuning, resulting in execution speeds roughly **150** times faster for Multinomial logistic regression and **50** times faster for Ordinal logistic regression than the pure Python autograd engine.

3. Convergence and Final Performance

Despite utilizing entirely different optimization strategies, both models successfully learned the underlying distribution, though they exhibited expected variations in their final convergence points. Evaluating the models on a dummy dataset revealed a maximum absolute difference between the learned feature weights of 0.2988. Furthermore, the Mean Absolute Error (MAE) of the final probability predictions between the two models was 0.0735.

This slight variation perfectly highlights the difference between the algorithms: the custom autograd engine relies on basic gradient descent with a fixed learning rate and epoch limit, meaning it was still incrementally approaching the exact minimum when training stopped. In contrast, Scipy's L-BFGS-B algorithm dynamically scales step sizes to pinpoint the exact global minimum almost immediately. Nevertheless, a prediction MAE of ~ 0.07 confirms that both the custom computational graph and the Scipy minimizer were structurally sound and functionally equivalent in traversing the loss landscape.

Part 2: Application & Analysis

1. Basketball Shot Selection Analysis (Part 2.1)

To analyze the factors influencing shot selection, I trained the custom `MultinomialLogReg` model on the provided basketball dataset. To establish a meaningful baseline, I forced the “above head” (standard jump shot) to be the reference class. Analyzing the resulting weights and p-values yielded several statistically significant insights:

Table 1. Significant Features (vs. Above Head)

Target	Feature	Coef (β)	SE	p-value	OR
Layup	Distance	-3.46	0.10	< 0.001	0.03
Layup	Movement_drive	19.70	0.25	< 0.001	3.6e8
Hook Shot	Distance	-1.84	0.08	< 0.001	0.16
Hook Shot	PlayerType_G	-1.66	0.13	< 0.001	0.19
Dunk	PlayerType_G	-3.05	0.18	< 0.001	0.05
Tip-In	PlayerType_G	-2.73	0.25	< 0.001	0.07

- **The Impact of Distance:** Across alternative shot types (Layup, Hook Shot, Dunk, Tip-In), the feature

Distance had a highly significant negative coefficient ($p < 0.001$). For example, as distance from the hoop increases, the odds of attempting a Layup (OR = 0.03) or a Hook Shot (OR = 0.16) are very small compared to an Above-Head shot. Practically, this means that players far from the basket will almost exclusively default to a standard jump shot rather than attempting specialized close-range mechanics.

- **Player Position Constraints:** The `PlayerType_G` (Guard) feature showed strong negative coefficients when predicting power-based or close-range shots like Dunks (OR = 0.05), Tip-Ins (OR = 0.07), and Hook Shots (OR = 0.19). This logically aligns with basketball reality: guards are typically shorter players positioned along the perimeter, making them significantly less likely to execute these interior shots compared to Centers (the implicit baseline).
- **Movement Dynamics:** The feature `Movement_drive` showed a massive positive coefficient for Layups ($\beta = 19.70, p < 0.001$). A player actively driving towards the basket is almost exclusively going to attempt a layup in motion rather than abruptly stopping to take a static above-head shot.

Conclusion on Model Logic: The abundance of highly significant coefficients that perfectly mirror real-world basketball mechanics demonstrates that multinomial logistic regression

is an exceptionally effective and highly interpretable model for inferring the dynamics of shot selection.

2. DGP: Ordinal vs. Multinomial Regression

To demonstrate a scenario where ordinal logistic regression definitively outperforms its multinomial counterpart, I constructed a Data Generating Process (DGP) simulating disease progression. The DGP utilized a latent continuous variable bounded by thresholds, intentionally generating a severe class imbalance: 609 Mild, 362 Moderate, and only 29 Severe cases.

When evaluated specifically on the rare **Severe** class, the **multinomial model failed completely**, correctly identifying 0 out of 29 cases. Because a multinomial model is greedy attempting to learn a completely independent set of feature weights for every target class the 29 samples were insufficient to estimate reliable weights, leading to misclassification.

Conversely, the **ordinal model successfully identified 3 of the 29 severe cases** and yielded a significantly higher average predicted probability for the true severe class (0.228 vs 0.160). This occurs because ordinal regression leverages the **proportional odds assumption**. It learns a single, shared set of feature weights across all 1,000 patients, borrowing statistical power from the majority classes to understand the general trajectory of the disease. It then only utilizes the 29 severe patients to calculate a single scalar threshold (intercept), making it vastly more robust when predicting rare, ordered categories.